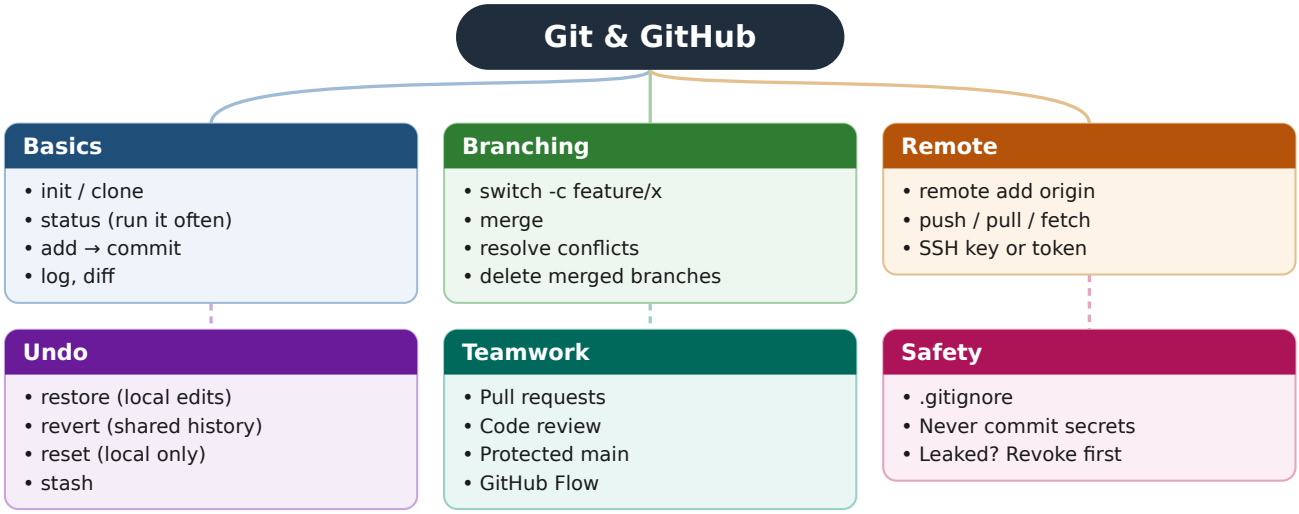
4. Git & GitHub

DevOps, Cloud & Platform Engineering Notes · Section 4 of 18 · Thirupathi Keloth · v1.4

In this section: 4.1 Concepts · 4.2 The three areas · 4.3 Everyday commands · 4.4 Branches and merging · 4.5 Remotes · 4.6 Undoing safely · 4.7 .gitignore and secrets · 4.8 Workflows · 4.9 More Git you will use at work



4.1 Concepts

Term	Meaning
Version control	Records every change so you can see history, undo and work together
Git	Version control tool that runs on your computer
GitHub	A website that hosts Git repositories online, plus pull requests, Actions and more
Repository (repo)	A project folder tracked by Git (the `.git` folder holds its history)
Commit	A saved snapshot with a message, author and unique ID (hash)
Branch	An independent line of work. `main` = the main branch
HEAD	Points to the commit you currently have checked out
Remote	A copy of the repo somewhere else, usually GitHub (`origin`)
Merge	Combine one branch into another
Pull request (PR)	A request to merge a branch, with review and checks
Merge conflict	Both sides changed the same lines; a person must decide

4.2 The three areas

```
Working directory  ──git add──►  Staging area  ──git commit──►  Repository (history)
   (your edits)                (next snapshot)                          │
                                                          git push ──►  GitHub (remote)
                                                          git pull ◄──
```

4.3 Everyday commands

Command	What it does
`git config --global user.name "Name"`	Set your name (once)
`git config --global user.email "you@mail.com"`	Set your email (once)
`git init`	Start tracking a folder
`git clone <url>`	Copy a remote repo to your computer
`git status`	What changed / what is staged. Run it often
`git add file` / `git add .`	Stage changes
`git commit -m "Add login page"`	Save a snapshot
`git log --oneline --graph`	Compact history
`git diff` / `git diff --staged`	Unstaged / staged changes
`git show <hash>`	Show one commit

4.4 Branches and merging

Command	What it does
`git branch`	List branches
`git switch -c feature/login`	Create and switch to a new branch
`git switch main`	Switch branch
`git merge feature/login`	Merge that branch into the current one
`git branch -d feature/login`	Delete a merged branch

Resolving a conflict: open the file, find the markers, keep the right code, delete the markers, then `git add` and `git commit`.

```
<<<<<<< HEAD
your version
=======
their version
>>>>>>> feature/login
```

4.5 Remotes

Command	What it does
`git remote -v`	Show remotes
`git remote add origin <url>`	Connect a local repo to GitHub
`git push -u origin main`	Upload and set upstream (first time)
`git push`	Upload commits
`git pull`	Download and merge remote changes
`git fetch`	Download without merging

GitHub authentication: SSH key (`ssh-keygen` , add the `.pub` to GitHub → Settings → SSH keys) or a personal access token. Test with `ssh -T git@github.com` .

4.6 Undoing safely

Situation	Command
Discard unstaged changes in a file	`git restore file`
Unstage a file	`git restore --staged file`
Fix the last commit message (not pushed yet)	`git commit --amend`
Undo a pushed commit safely (adds a new commit)	`git revert <hash>`
Move the branch back (local only, rewrites history)	`git reset --soft/--hard <hash>`
Save unfinished work temporarily	`git stash` , then `git stash pop`

Rule: never rewrite history (`reset` , `--force` push) on shared branches. Use `revert` .

4.7 .gitignore and secrets

```
# .gitignore
.env
*.pem
*.key
node_modules/
__pycache__/
.terraform/
*.tfstate
*.log
```

If a secret is committed: 1) revoke/rotate it immediately (it is compromised), 2) remove it from the code, 3) clean the history if needed, 4) add it to `.gitignore` , 5) use secret scanning (gitleaks, GitHub secret scanning).

4.8 Workflows

Workflow	How it works	Used by
GitHub Flow	`main` is always deployable; short feature branches → PR → merge	Most teams, simple
Trunk-based	Very short branches, merge to `main` many times a day, feature flags	High-speed teams with strong CI
GitFlow	`main` + `develop` + feature/release/hotfix branches	Scheduled releases, older style

Good commits: small, one purpose, clear message in the imperative mood ("Add health check endpoint"). Conventional style: `feat:` , `fix:` , `docs:` , `chore:` . **Tags:** `git tag v1.0.0` + `git push --tags` mark releases. **Protected branches:** require PR review and passing checks before merging into `main` .

4.9 More Git you will use at work

Command	What it does
`git rebase main`	Replay your branch's commits on top of the latest `main` (linear history). Only on your own, unshared branches
`git pull --rebase`	Update your branch without creating merge commits
`git cherry-pick <hash>`	Copy one commit onto the current branch (e.g. a hotfix)
`git blame file`	Who last changed each line, and in which commit
`git log -p file` / `git log --author=name`	History of a file / commits by an author
`git diff main...feature`	What a branch changes compared with `main`
`git tag -a v1.2.0 -m "Release 1.2.0"`	Annotated release tag
`git clean -fd`	Delete untracked files (careful)

Merge vs rebase: merge keeps the real history and adds a merge commit; rebase rewrites your commits into a straight line. Teams choose one style; **never rebase shared branches**. **Semantic versioning:** `MAJOR.MINOR.PATCH` → breaking change . new feature . bug fix (e.g. 2.4.1).

REMEMBER

- The flow: **edit → add → commit → push** (working dir → staging → repo → GitHub).
- Shared branch? Use **revert**, never reset or force-push.
- Small commits, clear messages: "Add health check endpoint".